
Project Scope, Objectives, and Literature Review Report

Project #43: Reconfigurable Neural Processing Unit (NPU) for Energy-Efficient AI at the Edge

Kristelle Sampang

Department of Electrical, Computer, and Software Engineering
The University of Auckland, Auckland, New Zealand

Tuesday 6 May 2025



Supervisor: Morteza Biglari-Abhari

Co-supervisor: Maryam Hemmati

Project Partner: Pratham Chhabra

***Disclaimer** – All these conference papers have been submitted as partial fulfilment for the project requirement for the BE(Hons) degree. Although they have been assessed, no errors or factual information have been corrected or checked.*

ABSTRACT

The increasing demand for Artificial Intelligence (AI) at the edge has introduced challenges in implementing Deep Neural Networks (DNNs) on resource-constrained embedded systems. These models require notable computational power and energy, making current designs unfit for edge devices. The literature review section explores existing research on energy-efficient AI acceleration, focused on Neural Processing Units (NPUs) implemented on Field Programmable Gate Arrays (FPGAs). The key findings consists of exploring techniques to reduce energy consumption without compromising accuracy, systolic implementation for MAC operations, and NPU performance ate edge. The benefits and limitations provide gaps that this project can investigate. This guides the project in the direction of developing a reconfigurable, energy-efficient NPU on FPGA for real-time AI inference at edge.

Contents

1	Literature Review	4
1.1	Introduction	4
1.2	Machine Learning	4
1.3	Energy-Efficiency	5
1.4	Neural Processing Unit	5
1.5	Field Programmable Gate Arrays	6
1.6	Systolic Arrays	7
2	Project Scope and Objectives	9
2.1	Project Statement	9
2.2	Research Problems	9
2.3	Aim and Objectives	9
2.4	Scope	9
2.5	Gantt Chart	10

1 LITERATURE REVIEW

1.1 Introduction

In recent years, there has been a rapid demand for Artificial Intelligence (AI) based applications. As these applications are computationally complex and power-hungry, researchers have investigated and developed hardware accelerators, such as Neural Processing Units (NPUs), to optimise performance whilst considering energy efficiency and accuracy. However, many of these accelerators are designed for high-performance systems that cannot easily adapt to edge devices with limited resources.

This literature review delves into existing research to support the development of a reconfigurable hardware accelerator implemented on a Field Programmable Gate Array (FPGA) tailored for low-power AI inference at edge. It focuses on opportunities for improving computational efficiency, how different architectures influence energy consumption, and discusses limitations of pre-existing implementations. By identifying these methods, this review provides a guideline for the project — an energy-efficient reconfigurable NPU at the edge.

1.2 Machine Learning

Machine Learning (ML) is a subset of AI that focuses on developing algorithms capable of learning from a given data set and improving performance without human intervention. A prominent type of ML model is Deep Neural Networks (DNN), which aims to mimic how a human brain works.

In embedded systems, the most popular type of DNN implemented is Convolutional Neural Networks (CNN). CNN is most used for image processing and object recognition. Structured like other Neural Networks (NN), it consists of input, hidden, and output layers (“But what is a neural network? — Deep learning chapter 1 - YouTube”, n.d.).

The input layer is the initial layer that transmits the given data into the network. The hidden layer identifies relevant data, extracts crucial features and patterns, and passes it onto the next layer. A network may have multiple hidden layers that analyse the previous layer’s output. It continuously passes through all hidden layers, each becoming narrower in scope like a funnel. Once the output layer is reached, it determines the output, where the final prediction is made.

The limitation of traditional CNN is that it only accepts a specific input size. This becomes an issue in cases where images vary in pixel resolution. To address this, a study by K.S et al. (2022) adds Spatial Pyramid Pooling (SPP) at the final convolutional layer or before the fully connected layer. This allows the network to accept dynamic input sizes and generate a fixed length output, regardless of input, and executed by adding zero padding. So, instead of training a model for all the sizes needed, only one model needs to be trained. The results show that the CNN-SPP configuration maintains a 95% accuracy relative to a traditional CNN model, making it an effective method.

Since embedded systems are resource-limited, implementing DNN training models has become a challenge. The training requires intense floating-point computations, back propagation, and weight gradient that strains the system’s power consumption (Wang et al., 2022). These requirements push the system past its capabilities and create a performance bottleneck. (Oh et al., 2017) As a result, training a DNN model is typically performed on a CPU and GPU and employed in an embedded environment (Oh et al., 2017). In some cases, models are transferred onto an online platform, like a cloud, but this creates a dependency that causes concern for data privacy and high power consumption during transmission (B. Kim et al., 2020). To address these limitations, researched has explored local optimisation strategies on embedded devices.. For example, B. Kim et al. (2020) proposed pipelined hardware scheduling to reduce idle time between CPU and accelerator, improving real-time performance and energy efficiency without relying on off-device processing.

1.3 Energy-Efficiency

One of the significant challenges of using Machine Learning is its high power consumption. As energy sources become limited, especially in edge devices, energy-efficiency becomes an important aspect of the system. A study by Gorvadiya et al. (2025) lists three key methods on how this can be performed.

- **Bit Quantisation** refers to representing the weights and activations of the models using lower-precision data types like INT8, FP16, and FP32. The study found that INT8 decreases memory usage the most and is effective for edge devices that support 8-bit operations. Conversely, FP32 is unsuitable for edge devices if resources are scarce. Therefore, FP16 balances the trade-off between efficiency without sacrificing too much accuracy.

Because large quantities of instructions occur in one AI operation, individual instructions do not need high precision, so redundant precision can be excluded. Furthermore, in a neural network, each neuron completes a small task relative to the large task. This means that if a neuron makes a mistake, it is minute, and the other neurons can compensate to maintain accuracy and produce a reasonable prediction. (“What is low-precision computing? - IBM Research”, n.d.)

- **Model Pruning** removes unnecessary or duplicated data. It focuses on removing data that do not significantly contribute to the final prediction. S. Kim et al. (2021) found that half the weights in convolutional layers can be set to zero without impacting the overall accuracy. To decide what should be pruned, it considers the sensitivity of the data and how it can influence the output effort. The results of Weight Pruning displayed that the impact on accuracy was only 1-2% but saved 35% on power. These results highlight the potential value of incorporating model pruning into the project. While pruning is typically applied during the training phase in software, this does not limit its relevance. Structured pruning is known to be suitable and well-supported in hardware as it produces regular patterns.
- **Adaptive Switching Techniques** exploit the ability to change the data type and precision used depending on the given task. This means it will maintain high accuracy on dominant operations when needed but switch to lower accuracy to save energy on less critical operations. This is powerful because it can dynamically switch to meet the resource constraints. Particularly for edge devices because it dynamically adapts to meet the constraints like application, load, and energy availability that change over time.

To measure energy efficiency, Guha and Chakrabarti (2024) calculates the throughput-to-power consumption ratio as a metric. A higher ratio indicates a more efficient system. This metric provides a useful benchmark that can be applied to assess the developed design in this project.

1.4 Neural Processing Unit

A Neural Processing Unit (NPU) is a specialised hardware accelerator, best at executing AI-based algorithms. Typically, NPUs function within a heterogeneous system with a CPU, in some cases a GPU, and sometimes an FPGA in embedded systems. In heterogeneous systems, accelerators become context-aware in different applications to improve computer efficiency. During run-time, task execution is assigned to the most appropriate core. This means only suitable NPU tasks must be offloaded onto the NPU, and the rest should be loaded onto the CPU. NPUs are best at performing high-volume yet computationally simple tasks, whereas CPUs are better at handling complex control logic. Because the system’s task assignment is efficient, completion time is reduced, leading to less energy consumed. (Liu et al., 2024)

A gap found by B. Kim et al. (2020) mentions a significant idle time in a CPU core (71%) and accelerator (29%) during runtime in a heterogeneous system. This wastes the limited resources that

embedded edge devices have. A pipelined idea was proposed where the CPU reads an input and offloads the neural-related computations onto the accelerator. Whilst the accelerator is busy handling the computations, the CPU will read another input to stay busy. This minimises the idle time as no waiting occurs between the CPU and the accelerator. Results of this case study show that the execution time of DNNs is reduced by 17.6% due to pipelining the computations.

NPU are notable for their fast computing parallelism. This means that tasks that are difficult to execute in parallel become a limitation. For example, Fast Fourier Transforms (FFT) operations are non-ideal on an NPU because they require a non-sequential memory access pattern, leading to slower data movement. Memory access operations, especially when located off-chip, like read or write, are complex and have high operating costs that make them inefficient. Furthermore, if a substantial amount of data transfer across processing elements is required, then inter-core data exchange cost has to be considered. This introduces latency, wastes energy, and requires extra communication (like a bus) between cores. Lastly, when equations consist of real and imaginary parts, the components must be operated on, processed, and stored separately. This doubles memory usage, which adds to the cost of memory (Liu et al., 2024). B. Kim et al. (2020) found an issue where low-precision FP16 operations by the NPU cause numerical overflow. This becomes an essential consideration if such low precision is worth the overflow issue.

1.5 Field Programmable Gate Arrays

Field Programmable Gate Arrays (FPGAs) are highly versatile in both functionalities and capabilities. In the papers reviewed, it has served two primary key roles: as a processing element in heterogeneous computing systems and as an implementation platform for designing Neural Processing Units (NPU) accelerators.

A case study by Liu et al. (2024) explores using a CPU, GPU, FPGA, and an NPU in a mixed configuration to test its performance. It found that GPUs highly accelerate computationally intensive tasks with parallelism at ease but with a trade-off of high power consumption (Oh et al., 2017). Instead, it utilises FPGAs to achieve energy-efficiency by consuming half the power of a GPU (Liu et al., 2024). Whilst the FPGA performed best in combination with a CPU for image processing, the same study demonstrated that a CPU/NPU pair configuration is best for AI models.

Similarly to the direction of this research, many of the studies reviewed have also explored implementing the NPU accelerators (Liu et al., 2024) and CNN on an FPGA. Some of the ideas discovered include:

- **Activation Function Unit**, used for determining whether or not a neuron should be activated, can be implemented using programmable Look Up Tables (LUT) which are FPGA friendly (Que et al., 2023).
- **Remarn**, implemented by Que et al. (2023), partitions a monolithic accelerator into multiple identical sub-accelerator cores, each functioning as a complete NPU. This allows spatial co-executions (executing multiple tasks simultaneously on separate hardware units) to increase throughput. Since smaller ML models may not fully utilise large monolithic cores, sub-cores can be sized appropriately to improve hardware utilisation and reduce hardware waste.
- **Vector Processing Block (VPB) and Convolution Processing Block (CPB)** pipelined together to create a pointwise and depthwise chaining method. This allows more than one CNN layer to be executed without saving the results to the off-chip memory, reducing the latency. Furthermore, it uses a CPU as a host processor to be in charge of the task control unit that supervises the data management of the NPUs, which are all connected through dedicated data channels (S. Kim et al., 2023).

- A **CNN Implementation** uses Embedded Memory Blocks (EMBs) for high-speed data access. The available memory blocks are loaded with pre-computed data, so fewer operations are carried out, leading to less latency and power used. Moreover, operations involving weights and biases are stored directly in the EMBs, eliminating transfers between the process and the memory, reducing the risk of theft by attackers, and enhancing the overall security of the implementation. (Guha & Chakrabarti, 2024)

A study by Beasley et al. (n.d.) demonstrates how Dynamic Partial Reconfiguration (DPR) is designed on an FPGA through Reconfigurable Regions (RR) to dynamically reprogram the process at runtime. RR restricts the compiler from using a region on the FPGA fabric so that Partial Reconfiguration Modules (PRM) can have designated space. During run time, the controller assigns the PRM used in a particular configuration through an initialised .rbf file, creating the advantage of swapping hardware acceleration on demand. However, this introduces trade-offs such as design complexity and under-utilisation if RR remains unused if no PRM is allocated. Changing configurations can take around hundreds of milliseconds, suitable for occasional task switching but less ideal for fast context switching. This design enables FPGAs to operate like multitasking processors for flexibility but context switching time can be improved on.

Whilst prior demonstrations provide insightful methods of implementing an NPU and CNN on FPGAs, some issues arise. Many rely on a fixed processor tailored for specific models or tasks that limits flexibility. A huge gap in this field contains an efficient implementation of a dynamically reconfigurable NPU that can adapt at runtime to changing parameters such as model size, performance demands, and power consumption. This project addresses this challenge by exploring the use of DPR to implement a modular energy-efficient NPU on FPGA.

1.6 Systolic Arrays

A systolic array is a structured network of processing elements (PEs) where computations are carried out in a systematic approach similar to pipelining. Each PE will receive, process, and pass on data to its neighbour concurrently in one clock cycle. Due to this, it provides characteristics like parallel computing, pipelining, synchronicity, and spatial and temporal locality. These are advantageous as processes are completed simultaneously at higher speeds, computations are timed by a global clock for predictable data movement, and faster memory accessing time. Additionally, systolic arrays are highly compact together and allow simple data control flow. As they are arrays, they can be scaled for larger data sets, perfect for ML, and the predictable structure makes it easier to design and optimise algorithms. Since the PEs are densely packed together, there is less data transmission, causing lower energy consumption. However, there are some disadvantages to systolic arrays. These include difficult and costly to build and specialised and inflexible in the problems it can solve, limiting its versatility. (“Parallel processing – systolic arrays — GeeksforGeeks”, n.d.) They are highly optimised for neural networks and matrix multiplications but have difficulty dealing with sparse matrices. Because of this, systolic arrays are not commonly implemented for sparse matrices.

To overcome this problem, an experiment conducted by He et al. (2020) demonstrates how systolic arrays can adapt to sparse matrices. The problems stem from inefficient MAC operations on PEs containing zero as the operations are performed by having no contributions. MAC operations are used because the operations in a convolutional layer of a DNN can be represented as

$$O[h][w][c] = \sum_{i=1}^{f_h} \sum_{j=1}^{f_w} \sum_{k=1}^{i_c} I[h + i \cdot s_h][w + j \cdot s_w][k] \times F[i][j][k][c]$$

so it would be ideal for optimising ML applications. The approach continues to use MAC to handle the matrix but a novel packing algorithm was applied. This merges columns that allow collisions to

occur so the amount of zeros that occur is reduced. This method found an energy saving of 4.39x for the INT8 implementation and a 19.79x for the FLOAT32 implementation. These results are relevant to this project as it showcases how negatively impacted energy consumption is when a large amount of PEs containing zeros are present. The issue with this method is that it already knows the sparse pattern before the matrix is loaded into the systolic array. For an input dependent systolic array, like a dynamic sparse pattern, this method cannot be applied.

Another case study, by Choi et al. (2023), found the current NPU and systolic array implementations have low hardware utilisation when only one model is run. To address this issue, Data-Mirroring NPU (DM-NPU) was proposed. This architecture functions by splitting the systolic array, for example in half. One half of the array will have a normal data flow whereas the other half will have a "mirrored" or opposite data flow. This means if the first half is going from left to right and top to down, the opposing data flow will be right to left and down to top. Because of this change, if two models are running, it avoids the second model from polling or waiting for the first model to use the PEs. Additionally, more PEs are used up in one clock cycle so better hardware utilisation is achieved. Planaria, implemented by Ghodrati et al. (2020), is another method of enabling multiple models running on shared systolic arrays. This addresses the same issue with low hardware utilisation as Choi et al. (2023) via partitioning the array. These implementations prove that resolving the sparse-matrix issue is feasible by improving upon it. This pushes the project to take on a direction towards utilising the systolic array's power-efficiency.

2 PROJECT SCOPE AND OBJECTIVES

2.1 Project Statement

The research project aims to design and implement a reconfigurable Neural Processing Unit (NPU) on FPGA. By investigating the computation complexity of machine learning algorithms and building upon existing architectures, the system minimises energy consumption for AI applications on edge devices.

2.2 Research Problems

The literature review has highlighted several significant gaps that need further investigation:

- Limited resources on edge device hinder training on the embedded system. Methods to reduce machine learning complexity must be executed outside the device.
- Bit quantisation inference risks numerical overflow, especially for low-precision data types like INT8 and FP16, that lowers accuracy.
- Dynamic sparsity patterns in matrices are under explored in systolic array implementations. Investigate techniques to utilise on how to effectively exploit sparsity to reduce unnecessary MAC operations.
- Lack of NPU implementation on FPGA on edge devices.

2.3 Aim and Objectives

The aim is to design, implement, and evaluate a reconfigurable NPU on FPGA that is optimised for low-power, high throughput of CNN models, and to implement found research to aid this.

- Train CNN model based on identified optimisation techniques, evaluate its impact on potential overflow, and execute on FPGA at edge level.
- Implement and deploy an object recognition CNN model on the FPGA to emulate an edge-device.
- Design and implement hardware NPU datapath on FPGA, focused on systolic arrays for MAC operations integrated with embedded memory blocks.
- Implement improved hardware utilisation of systolic arrays through techniques of mirroring and adapting to sparse matrices.
- Describe reconcilability of NPU and evaluate performance based on key metrics such as execution time and power consumption.
- Perform benchmark testing to measure execution time, throughput, and power consumption of the system to improve performance through iterations.

2.4 Scope

This research project will be conducted under a strict time constraint of two university semesters, which limits the depth of the experimentation. The main goal is to complete at least one working

iteration of a reconfiguration NPU implemented on an FPGA. The NPU will be based on a systolic array architecture, with reconfigurable data precision.

The project will only use pre-trained CNN models to reduce the workload. Benchmark metrics will include execution time, performance, throughput, and power consumption.

Hardware development will be completed on either DE1-SOC or DE2-115 development boards as the team has prior experience. However, previous challenges with the software (e.g. Quartus and NIOS II) have hindered work progression due to crashes and instability, which will have to be managed.

Task responsibilities will be divided and allocated based on the member's strengths. All members will work on all areas of the project but specialised tasks will be delegated. For example, under the energy-efficiency topic, one member can focus on machine learning and the other can focus on NPU power consumption. This ensures all aspects of the project will be covered in the given time frame.

2.5 Gantt Chart

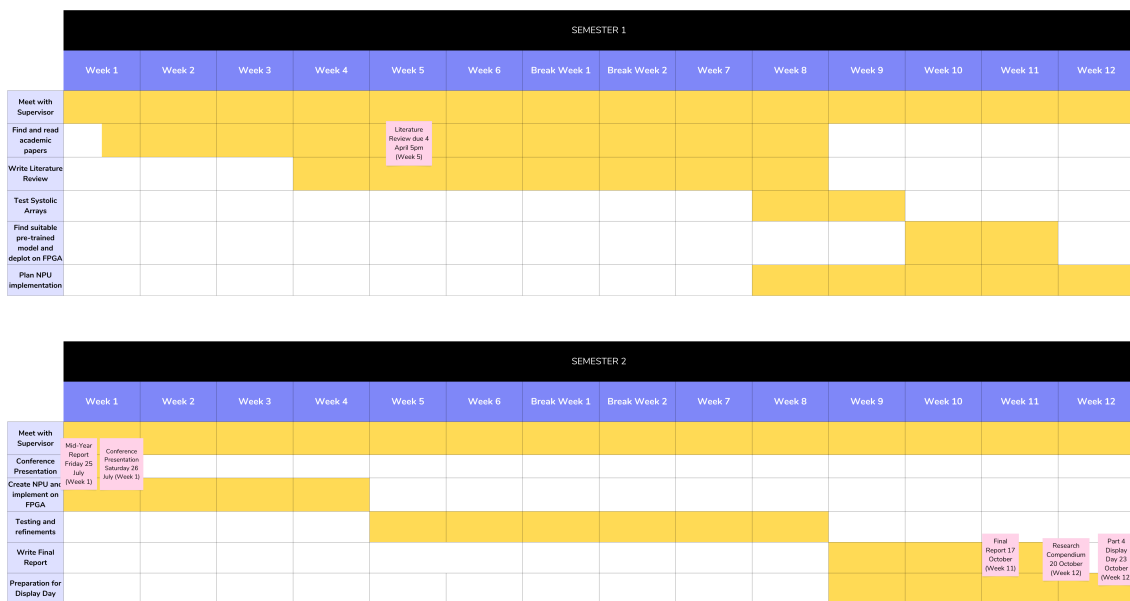


Figure 1: Gantt Chart for Semester 1 and Semester 2

If the project does not go according to plan in terms of timing, the inter-sem will be used to get up to date.

References

- Beasley, A., Walker, L., & Clarke, D. C. (n.d.). Developing and Implementing Dynamic Partial Re-configuration for Pre-Emptible Context Switching and Continuous End-To-End Dataflow Applications.
- But what is a neural network? — Deep learning chapter 1 - YouTube. (n.d.). Retrieved April 2, 2025, from https://www.youtube.com/watch?v=aircAruvnKk&ab_channel=3Blue1Brown
- Choi, J., Ha, Y., Lee, J., Lee, S., Lee, J., Jang, H., & Kim, Y. (2023). Enabling Fine-Grained Spatial Multitasking on Systolic-Array NPUs Using Dataflow Mirroring. *IEEE Transactions on Computers*, 72(12), 3383–3398. <https://doi.org/10.1109/TC.2023.3299030>
- Ghodrati, S., Ahn, B. H., Kyung Kim, J., Kinzer, S., Yatham, B. R., Alla, N., Sharma, H., Alian, M., Ebrahimi, E., Kim, N. S., Young, C., & Esmaeilzadeh, H. (2020). Planaria: Dynamic Architecture Fission for Spatial Multi-Tenant Acceleration of Deep Neural Networks. *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 681–697. <https://doi.org/10.1109/MICRO50266.2020.00062>
- Gorvadiya, J., Chagela, A., & Roy, M. (2025). Energy Efficient Pruning and Quantization Methods for Deep Learning Models. *2025 International Conference on Sustainable Energy Technologies and Computational Intelligence (SETCOM)*, 1–6. <https://doi.org/10.1109/SETCOM64758.2025.10932458>
- Guha, K., & Chakrabarti, A. (2024). Secure Energy-Efficient Implementation of CNN on FPGAs for Accuracy Dependent Real Time Task Processing. *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 1–6. <https://doi.org/10.1109/ISVLSI61997.2024.00012>
- He, X., Pal, S., Amarnath, A., Feng, S., Park, D.-H., Rovinski, A., Ye, H., Chen, Y., Dreslinski, R., & Mudge, T. (2020). Sparse-TPU: Adapting systolic arrays for sparse matrices. *Proceedings of the 34th ACM International Conference on Supercomputing*, 1–12. <https://doi.org/10.1145/3392717.3392751>
- Kim, B., Lee, S., Trivedi, A. R., & Song, W. J. (2020). Energy-Efficient Acceleration of Deep Neural Networks on Realtime-Constrained Embedded Edge Devices. *IEEE Access*, 8, 216259–216270. <https://doi.org/10.1109/ACCESS.2020.3038908>
- Kim, S., Cho, S., Park, E., & Yoo, S. (2021). FPGA Prototyping of Systolic Array-based Accelerator for Low-Precision Inference of Deep Neural Networks. *2021 IEEE International Workshop on Rapid System Prototyping (RSP)*, 1–7. <https://doi.org/10.1109/RSP53691.2021.9806200>
- Kim, S., Jang, B., Lee, J., Bae, H., Jang, H., & Park, I.-C. (2023). A CNN Inference Accelerator on FPGA With Compression and Layer-Chaining Techniques for Style Transfer Applications. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(4), 1591–1604. <https://doi.org/10.1109/TCSI.2023.3234640>
- K.S, B. S., Fahmi, A., Adriansyah, N. M., & W. Prabowo, V. S. (2022). Modified CNN to Maximize Energy Efficiency in D2D Underlying with Multi-Cell Cellular Network. *2022 IEEE International Conference on Cybernetics and Computational Intelligence (CyberneticsCom)*, 359–364. <https://doi.org/10.1109/CyberneticsCom55287.2022.9865637>
- Liu, X., Xu, W., Wang, Q., & Zhang, M. (2024). Energy-Efficient Computing Acceleration of Unmanned Aerial Vehicles Based on a CPU/FPGA/NPU Heterogeneous System. *IEEE Internet of Things Journal*, 11(16), 27126–27138. <https://doi.org/10.1109/JIOT.2024.3397649>
- Oh, S., Kim, M., Kim, D., Jeong, M., & Lee, M. (2017). Investigation on performance and energy efficiency of CNN-based object detection on embedded device. *2017 4th International Conference on Computer Applications and Information Processing Technology (CAIPT)*, 1–4. <https://doi.org/10.1109/CAIPT.2017.8320657>
- Parallel processing – systolic arrays — GeeksforGeeks. (n.d.). Retrieved April 2, 2025, from <https://www.geeksforgeeks.org/parallel-processing-systolic-arrays/>
- Que, Z., Nakahara, H., Fan, H., Li, H., Meng, J., Tsoi, K. H., Niu, X., Nurvitadhi, E., & Luk, W. (2023). Remarn: A Reconfigurable Multi-threaded Multi-core Accelerator for Recurrent Neural

-
- Networks. *ACM Transactions on Reconfigurable Technology and Systems*, 16(1), 1–26. <https://doi.org/10.1145/3534969>
- Wang, Y., Deng, D., Liu, L., Wei, S., & Yin, S. (2022). PL-NPU: An Energy-Efficient Edge-Device DNN Training Processor With Posit-Based Logarithm-Domain Computing. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 69(10), 4042–4055. <https://doi.org/10.1109/TCSI.2022.3184115>
- What is low-precision computing? - IBM Research. (n.d.). Retrieved April 2, 2025, from <https://research.ibm.com/blog/low-precision-computing>